Fixing Over-Specificity in Concept Memory

From Problem Diagnosis to Scale Validation

gpt-5-nano → qwen3.5-flash | 20-problem iteration → full-scale eval

2026-03-10

The Problem: Over-Specificity Kills Selection

The m3 experiment (gpt-5-nano, 200 math problems, 837 concepts) revealed:

- **76% of concepts** contained source-specific numeric values
- These leak into hints for *different* problems, causing contradictory guidance

Case: cmath_5298 (highway car spacing)

```
- concept: Floor-Ceiling Interval Bound for Integer Counting
  implementation:
    - n = 1 + 23k, enforce -9999 ≤ n ≤ -1000     # ← leaked values
    - k_max = floor((-1001)/23), n = 1+23·k_max = -1011
```

These numbers come from the **source** problem, not the target. The model anchors on "floor(b) - ceil(a) + 1", missing a boundary case. Baseline: 4/4. Concept: *1/4*.

Net m3 effect: +4.3 p1 uplift, but ~2 harms per run from leakage and over-matching.

Three Proposed Solutions

From channel feedback (Matthew Ho, 2026-03):

1. Dev set concept rejection

Use a dev set to test if new concepts hurt downstream performance; reject bad ones.

2. Parameterization for generalization

Contain over-specific detail as typed parameters — slots to be filled, not constraints. Core to ArcMemo's design philosophy. Two sub-directions:

- **Numerical detail → parameters:** "grid has even dimensions" → `param: grid_dim, type: int`
- **Conceptual detail → functional params:** "modular arithmetic with prime modulus" → `param: modulus_type, type: str`

3. Changing the "embedding" (selection presentation)

Force over-specific detail into sub-attributes hidden from the selector. If the selector can't see implementation details, it can't over-match on them.

What We Tried

Three extraction variants, three selector configurations, 20-problem fruit fly sets

Attempt 1: v3a Anti-Leakage Extraction

Target: Solution #1 (dev set) + Solution #2 (parameterization, partially)

Changed the extraction prompt to explicitly forbid source-specific values:

```
BAD:  "compute 160mi / 5hr = 32mph"
GOOD: "compute distance ÷ time to get speed"

BAD:  "base CM = 2, height = 8 → area = 8"
GOOD: "identify base and height → compute triangle area"
```

Added to prompt: *"Do not include specific values from the source problem. Describe the general procedure abstractly."*

Leakage results

	Before (v1/v2)	After (v3a)
Math concepts with source values	76% (634/837)	0% (0/53)
LCB concepts with source values	<1% (1/168)	0% (0/42)

Attempt 2: v3c Parameterization Enforcement

Target: Solution #2 — force all specifics into typed parameters.

Made `parameters` required in extraction. Added "Parameterization over specificity" instructions pushing variable aspects into parameters instead of cues.

v3c extraction stats

- Math: 49 concepts, **0 without parameters** (vs 8/53 in v3a)
- LCB: 41 concepts, **0 without parameters**

v3c solve results (20-problem sets, 2 seeds)

Domain	v3a	v3c	Delta
Math	**20.0**	19.0	*-1.0*
LCB	**18.5**	17.0	*-1.5*

Verdict: worse on both domains. Reverted. When forced to parameterize everything, the model strips useful specificity from cues. Parameters are hidden from the selector — so it sees only the now-too-generic cues.

Attempt 3: Two-Tier Rendering (Selector ≠ Solver)

Target: Solution #3 — hide detail from the selector.

New `selector_render_mode` config: show different concept fields to selector vs solver.

Field	`full` (solver)	`cues_only` (selector)	`name_only`
name	✓	✓	✓
description	✓	✓	✗
cues	✓	✓	✗
implementation	✓	✗	✗
parameters	✓	✗	✗

Two-Tier Rendering: Results (20-problem sets, 2 seeds)

Math:

Selector	Solver	Mean
full	full	19.0
cues_only	**full**	**20.0**
name_only	full	19.0
cues_only	cues_only	20.0

LCB:

Selector	Solver	Mean
full	**full**	**18.5**
cues_only	full	17.5
full	cues_only	*14.5*

Math optimal: cues_only selector, full solver — **20.0**
LCB optimal: full selector, full solver — **18.5**

Why Domains Diverge

Math is name-driven

"Vieta's Formulas" tells the solver exactly what to do. Implementation details in the selector just cause over-matching on procedural keywords like "polynomial" or "quadratic".

Hiding implementation from the selector: +1.0 (19.0 → 20.0)

Code is procedure-driven

"Digit DP" alone is insufficient. The solver (and selector) need to know about tight-bound tracking, state representation, modulo handling to distinguish it from "1D DP State Compression".

Hiding implementation from the selector: -1.0 (18.5 → 17.5)
Hiding implementation from the solver: -4.0 (18.5 → 14.5)

v3b: Drop Implementation Entirely

Also tested a lighter extraction (v3b) that keeps only name + description + cues. Hints are 40% shorter.

Domain	Baseline	v3a	v3b
Math	19.0	19.0	**20.0**
LCB	16.0	**18.5**	16.0

v3b = baseline on LCB. Without implementation details, code concepts carry no actionable information for the solver. **Rejected** as a general solution — v3a with two-tier rendering achieves the same math benefit without sacrificing LCB.

The Winner: v3a + Domain-Specific Selection

Validated on 20-problem fruit fly sets, then scaled to full eval

Winning Configuration

Setting	Math	LCB
Extraction prompt	v3a (anti-leakage, abstract impl)	v3a
Selector render mode	cues_only	full
Solver render mode	full	full
Retry mode	hybrid	concept

What each solution contributed

Proposed solution	Approach tested	Outcome
Dev set rejection	20-problem fruit fly iteration	Adopted — rapid testing loop
Parameterization (v3c)	Force params in extraction	Rejected — hurts both domains
Change embedding (two-tier)	`selector_render_mode`	Adopted for math — cues_only selector
(additional) Anti-leakage	v3a extraction prompt	Adopted — 76% → 0% leakage

Scale: Full Pipeline

Full pipeline at scale: build → extract (v3a) → select (domain-specific) → eval

	LiveCodeBench	Competition Math
Build set	200 problems	500 problems
Build solve rate	160/200 (80%)	485/500 (97%)
Concepts extracted	239	1105
Eval set	100 problems	200 problems (L5)
Selection coverage	**92/100 (92%)**	**131/200 (66%)**
Model	qwen3.5-flash	qwen3.5-flash
Passes	2 (p1 + retry)	2 (p1 + retry)

Scale Results: LiveCodeBench

100 eval problems, qwen3.5-flash, n=1, 2 passes

Config	Pass 1	Final	Retry recovery	vs Baseline
Baseline	74	80	6/26 (23%)	—
Concept v2 (old)	70	81	11/30 (37%)	+1
Concept v3a	**73**	**85**	**12/27 (44%)**	**+5**
Hybrid v3a	72	80	8/28 (29%)	0

+5 over baseline (+6.25%)

- v3a nearly eliminates p1 regression (74→73, just -1 vs v2's -4)
- Retry recovery **doubles**: 44% vs 23% baseline
- Concepts provide new algorithmic angles that help the solver escape initial failure
- v2 was barely positive (+1) — the leakage fix is what unlocked the +5

Scale Results: Competition Math

200 eval problems, qwen3.5-flash, n=1, 2 passes

Config	Pass 1	Final	Retry recovery	vs Baseline
Baseline	**196**	**197**	1/4 (25%)	—
Concept v3a	195	196	1/5 (20%)	*-1*
Hybrid v3a	193	194	1/7 (14%)	*-3*

Concepts hurt at ceiling (-1 to -3)

- Baseline at **98.5%** — only 3 problems unsolved
- Any hint noise is more likely to harm the 197 correct than save the 3 failures
- Selection coverage only 66% (1105-concept library overwhelms selector)

The Headroom Principle

	LCB	Math
Baseline	80%	98.5%
Available problems	20	3
v3a delta	+5	*-1*

Concepts help when there is room to improve

At 80% baseline, concepts recover 12/27 failures on retry — a genuine mechanism. At 98.5%, the upside is max +3, the downside is up to -197 from noise. The expected value is negative whenever hint noise rate exceeds ~1.5%.

Same domain, different headroom

The m3 experiment showed concepts help math at **+4.3 p1** with nano (87% baseline). Flash at 98.5% baseline: concepts hurt. The domain hasn't changed — the headroom has.

Selection Scaling: A New Bottleneck

Library size	239 (LCB)	1105 (Math)
Selection coverage	92%	66%
"None" responses	0	68

The selector matches concepts reliably from ~200 entries. At ~1100 concepts it returns "None" for 34% of problems. This held for both render modes:

Render mode	Math coverage
cues_only	131/200 (66%)
full	79/200 (40%)

`full` mode is *worse* — longer prompts cause more "None" responses.

Fruit fly experiments (50-100 concepts) showed no scaling concern. **This is a new bottleneck** discovered at scale.

Summary

What Worked, What Didn't

Approach	Result	Status
v3a anti-leakage extraction	76% → 0% leakage; LCB +5 at scale	**Adopted**
Two-tier rendering (cues_only selector)	Math 19.0 → 20.0 on fruit fly	**Adopted for math**
v3b drop implementation entirely	Math +1.0, *LCB = baseline*	**Rejected**
v3c forced parameterization	Math -1.0, LCB -1.5	**Rejected**
Dev set fruit fly iteration	Enabled rapid testing (20 problems, 2 seeds)	**Adopted as method**
Hybrid retry mode	Best final score with nano; no help at flash ceiling	**Domain-dependent**

The winning combination

v3a extraction + domain-specific selector render mode addresses over-specificity through two complementary mechanisms:

1. **Extraction side:** strip source-specific values from concepts
2. **Selection side:** hide procedural detail from selector (math only)

LCB Result: Validated at Scale

Config	Final	vs Baseline
Baseline	80	—
Concept v3a	85	+5 (+6.25%)

- First time concept augmentation shows meaningful gain at full scale
- Retry recovery doubles (44% vs 23%)
- v2 → v3a: leakage fix turned +1 into **+5**

Math Result: Ceiling Effect

Config	Final	vs Baseline
Baseline	197	—
Concept v3a	196	-1

- Baseline too strong (98.5%) for concepts to help
- Need harder problems or weaker model to test math concept effectiveness

Open Problems

1. Selection does not scale past ~300 concepts

66% coverage at 1105 concepts. Needs concept library pruning, clustering, or hierarchical selection.

2. Headroom requirement limits applicability

Concepts help at ~80% baseline, hurt at ~98%+. Need to characterize the crossover point more precisely.

3. Dev set concept rejection not yet automated

Fruit fly iteration was manual. The full rejection loop (extract → select → solve → attribute → reject) is not yet implemented.

4. Single-seed LCB result

The +5 finding is from one seed. Multi-seed validation needed to confirm robustness.

Next Steps

1. Multi-seed LCB validation (s43/s44)
2. Concept library pruning → re-evaluate math
3. Harder math benchmark (flash baseline ~80-85%)
4. Automated concept rejection pipeline